

SYSTEM REQUIREMENTS SPECIFICATION

ReMediX

Vascular Mapping and Analysis Platform

1. Introduction

1.1 Purpose

This System Requirements Specification (SRS) defines the functional and non-functional requirements for ReMediX, a web-based biomedical decision-support platform for drug repurposing research. The document provides a comprehensive specification that serves as a contract between stakeholders and the development team, establishing clear expectations for system capabilities, architecture, constraints, and design principles.

1.2 Document Conventions

- Functional Requirements are prefixed with 'FR-' followed by section and item number (e.g., FR-1.1)*
- Non-Functional Requirements are prefixed with 'NFR-' (e.g., NFR-1.1)*
- API endpoints are formatted as HTTP METHOD /path/to/endpoint*
- Technical terms are defined in the Glossary (Appendix A)*
- All measurements use standard SI units unless otherwise specified*
- 'Shall' indicates mandatory requirements; 'Should' indicates recommended features*

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Development Team: Review entire document with emphasis on sections 3 (System Features), 4 (External Interface Requirements), and 5 (Non-Functional Requirements)*
- Project Managers: Focus on sections 1 (Introduction), 2 (Overall Description), 6 (Other Requirements), and risk/constraint sections*
- Quality Assurance: Use sections 3, 4, 5, and Appendix C (Issues List) for test case development*

- *System Architects: Review sections 2.5, 4, and all non-functional requirements for design decisions*
- *Stakeholders/Product Owners: Start with section 2 (Overall Description) and section 3 (System Features) for functional overview.*

1.4 Project Scope

ReMediX is a single-deployable web application for biomedical researchers that combines compound identity lookup, pharmacology database integration, bioactivity analysis, and disease signature matching to generate drug repurposing scores. The system integrates heterogeneous data sources (PubChem, DrugBank, ChEMBL, CREEDS, GeneCards) and provides interactive interfaces for researchers to explore compound-disease relationships.

In Scope:

- *Compound search by name and SMILES notation*
- *Integration with external pharmacology and bioactivity databases*
- *Disease signature matching and gene target analysis*
- *Quantitative repurposing score computation*
- *Interactive web-based user interface with multiple data exploration views*
- *JSON and tabular data export functionality*

Out of Scope:

- *User account management and authentication (internal use only)*
- *Machine learning model development or training*
- *Mobile application development*
- *Database persistence layer for user jobs/results*
- *Real-time collaboration features*
- *Predictive analytics or forecasting*

1.5 References

- *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*
- *FastAPI Documentation: <https://fastapi.tiangolo.com>*
- *React 18 Documentation: <https://react.dev>*
- *PubChem API Documentation: <https://pubchem.ncbi.nlm.nih.gov/docs/>*

- ChEMBL Web Resource Client:
https://github.com/chembl/chembl_webresource_client
- DrugBank Database: <https://go.drugbank.com/>
- CREEDS Database: <http://www.maayanlab.net/CREEDS/>

2. Overall Description

2.1 Product Perspective

ReMediX operates as a standalone, integrated system that bridges the gap between biomedical data sources and researcher workflows. Rather than requiring researchers to query multiple databases separately, the platform provides a unified interface that combines compound identification, pharmacology lookups, and disease signature analysis in a single workflow.

The system is positioned as:

- A research tool for exploratory drug discovery and repurposing analysis
- A decision-support system that synthesizes disparate biomedical datasets
- A collaboration bridge between computational biologists and pharmacologists
- An internal platform for controlled, research-focused use (not a public-facing service)

2.2 Product Features

Core Features:

- *Compound Identity Resolution:* Search and retrieve compounds from PubChem by name or SMILES notation, with InChIKey normalization for cross-database matching
- *Integrated Pharmacology Lookup:* Retrieve curated drug properties and classifications from local DrugBank repository with efficient streaming XML parsing
- *Bioactivity Enrichment:* Fetch binding assay data (IC50, AC50, Ki values) from ChEMBL external API and extract associated target gene sets
- *Disease Signature Matching:* Load and match gene targets against curated CREEDS disease signatures with beneficial/harmful gene classification

- *Quantitative Scoring: Compute normalized repurposing scores based on gene-disease overlap using Excel-based lookup tables*
- *Interactive Data Exploration: Browse gene expression data, disease signatures, and dataset metadata through paginated interfaces*
- *Data Export: Support JSON export of match results, scores, and disease signature tables for downstream analysis*

2.3 User Classes and Characteristics

User Class	Characteristics	Key Needs
<i>Biomedical Researcher</i>	<i>PhD-level scientist with knowledge of drug repurposing concepts, gene targets, disease biology. Comfortable with computational tools and data analysis.</i>	<i>Fast compound-to-disease matching, interpretable scores, ability to export results for further analysis in specialized tools (R, Python)</i>
<i>Pharmacologist</i>	<i>Domain expert in drug properties, mechanisms of action, clinical applications. May have limited computational background.</i>	<i>Clear visualization of compound properties and bioactivity data, intuitive disease selection, reliable pharmacology information</i>
<i>Bioinformatician</i>	<i>Technical specialist with expertise in data integration, gene analysis, and systems biology. Expects programmatic access and data standardization.</i>	<i>API access, structured data formats (JSON), documented schemas, ability to integrate with custom analysis pipelines</i>
<i>Project Manager / Team Lead</i>	<i>Non-technical role overseeing research initiatives. Needs to track findings and coordinate across teams.</i>	<i>Clear reporting, exportable results, audit trail of analysis performed, team accessibility and reliability</i>

2.4 Operating Environment

2.4.1 Backend Environment

- *OS: Linux (Ubuntu 20.04 LTS or newer), macOS, or Windows Server with WSL2*
- *Python: Version 3.14 or newer*
- *Runtime: FastAPI with Uvicorn ASGI server*
- *Memory: Minimum 4GB RAM for local dataset caching (8GB+ recommended for concurrent users)*
- *Storage: Sufficient disk space for local datasets (DrugBank XML ~500MB, CREEDS JSON ~100MB, Excel files ~50MB)*
- *Network: Outbound HTTPS connectivity to PubChem and ChEMBL APIs*

2.4.2 Frontend Environment

- *Browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+ (ES2020 support required)*
- *Build Environment: Node.js 16+ with npm or yarn for development*
- *Build Tool: Vite (configured for production SPA deployment)*
- *Network: HTTP/HTTPS connectivity to backend API (default localhost:8000 or configurable)*

2.4.3 Development Environment

- *Version Control: Git (GitHub or similar)*
- *IDE Support: VS Code, PyCharm, WebStorm, or equivalent*
- *Testing: pytest (backend), Jest/Vitest (frontend) for automated testing*
- *Documentation: Markdown for API and project documentation*

2.5 Design and Implementation Constraints

- *Single-Process Deployment: System operates as monolithic single Python process; no microservices architecture or external job queue*
- *Local Data Storage: All biomedical datasets (DrugBank, CREEDS, GeneCards) must be pre-provisioned locally; no cloud database backend*

- *No User Persistence: System does not implement user accounts, authentication, or result persistence; each browser session is stateless*
- *Streaming XML Parsing: DrugBank XML must be parsed incrementally due to file size; full in-memory loading is not feasible*
- *File System Dependency: Disease signature table exports are written to local filesystem synchronously during request handling*
- *Fixed Normalization Constant: Repurposing score normalization uses hardcoded divisor (283.4119365); changes require code modification*
- *Python Version Lock: Explicit dependency on Python 3.14+; backward compatibility with earlier versions not maintained*

2.6 User Documentation

The following documentation will be provided:

- *Getting Started Guide: Installation, data setup, and first compound search walkthrough*
- *User Manual: Step-by-step instructions for each interface page and common workflows*
- *API Documentation: OpenAPI/Swagger specification with endpoint descriptions, parameters, and response examples*
- *Data Dictionary: Definitions of all fields in results and export files*
- *Troubleshooting Guide: Common error messages and resolution steps*
- *System Administrator Guide: Deployment, configuration, dataset updates, and monitoring*

2.7 Assumptions and Dependencies

2.7.1 Assumptions

- *Users possess PhD-level or equivalent scientific training in biomedical fields*
- *All local biomedical datasets (DrugBank, CREEDS, GeneCards) are legally licensed and pre-downloaded*
- *Internet connectivity is available for PubChem and ChEMBL API calls*
- *System is deployed in a controlled, internal research environment (not exposed to untrusted networks)*
- *User job results are not required to persist beyond a single browser session*

- Researchers have access to supplementary analysis tools (R, Python, Excel) for downstream data processing

2.7.2 Dependencies

- External: PubChem API availability for compound property lookups
- External: ChEMBL webresource API availability for bioactivity data
- Internal: Availability of local DrugBank XML, CREEDS JSON, and Excel data files on disk
- Internal: Correct file paths and read permissions for all local datasets
- Infrastructure: Python 3.14+ runtime and Node.js build environment for deployment
- Third-Party Libraries: FastAPI, React 18, openpyxl, lxml, pandas, chembl-webresource-client (versions specified in pyproject.toml and package.json)

3. System Features

3.1 Compound Lookup and Property Retrieval

3.1.1 Description

This feature allows researchers to search for chemical compounds by common name or SMILES notation and retrieve standardized compound properties for subsequent analysis.

3.1.2 Functional Requirements

- FR-1.1: System shall accept compound search by common name and return properties from PubChem (name, molecular weight, structure image, etc.)
- FR-1.2: System shall accept compound search by SMILES notation and resolve to PubChem data with matching confidence level
- FR-1.3: System shall compute and return InChIKey for all compounds to enable consistent cross-database matching
- FR-1.4: System shall handle invalid or ambiguous compound inputs gracefully with informative error messages and suggestions for valid input format
- FR-1.5: System shall cache PubChem results for 24 hours to reduce redundant API calls for frequently searched compounds

3.1.3 Interfaces

- UI: HomePage component with two input fields (name or SMILES) and search button
- API: GET /api/compound/name/{name}/properties and GET /api/compound/smile/{smile}/properties
- Data: Results displayed as structured panels with compound image, key properties, and action buttons

3.2 Pharmacology Data Integration

3.2.1 Description

This feature integrates local DrugBank and external ChEMBL datasets to provide comprehensive pharmacology and bioactivity information for identified compounds, enabling researchers to understand drug-target relationships.

3.2.2 Functional Requirements

- FR-2.1: System shall retrieve compound properties from local DrugBank dataset using InChIKey with streamed XML parsing
- FR-2.2: System shall parse DrugBank XML without loading entire dataset into memory by implementing incremental streaming
- FR-2.3: System shall retrieve bioactivity data from ChEMBL API for standard assay types (IC50, AC50, Ki, etc.)
- FR-2.4: System shall implement LRU caching for ChEMBL results to avoid redundant external API calls within 12-hour window
- FR-2.5: System shall extract target gene sets from bioactivity records and present deduplicated gene list
- FR-2.6: System shall handle cases where DrugBank or ChEMBL data is unavailable with graceful degradation and clear messaging

3.2.3 Interfaces

- API: GET /api/drugbank/inchikey/{inchikey}/properties (returns pharmacology data)
- API: GET /api/chembl/inchikey/{inchikey}/bioactivity?standard_type=IC50|AC50|Ki (returns bioactivity and genes)
- UI: Pharmacology panel displays drug classification, clinical uses (from DrugBank) and binding data (from ChEMBL)

3.3 Disease Signature Matching and Gene Classification

3.3.1 Description

This feature matches compound target genes against disease signatures from CREEDS, classifying genes as beneficial (up-regulated in disease) or harmful (down-regulated in disease) to enable scoring.

3.3.2 Functional Requirements

- *FR-3.1: System shall load CREEDS disease signatures from local JSON dataset at startup with validation*
- *FR-3.2: System shall support case-insensitive disease name lookup by normalizing input (strip whitespace, convert to lowercase)*
- *FR-3.3: System shall classify genes as beneficial if present in disease up-gene set, harmful if in down-gene set*
- *FR-3.4: System shall compute aggregate beneficial and harmful gene counts per disease*
- *FR-3.5: System shall generate and cache disease signature tables (up/down genes) for rapid access on repeated queries*
- *FR-3.6: System shall persist generated disease signature tables to local filesystem for long-term reference*

3.3.3 Interfaces

- *API: GET /api/diseases (returns all available disease names)*
- *API: GET /api/match?genes=gene1,gene2,...&disease=X (returns gene classification)*
- *API: GET /api/diseaseSignature/table?disease=X&page=0&page_size=50 (returns paginated signature table)*
- *UI: GeneMatchPage autocomplete for disease selection, results table showing beneficial/harmful counts*

3.4 Repurposing Score Computation

3.4.1 Description

This feature computes a quantitative repurposing score for each compound-disease pair based on the overlap of compound targets with disease-associated genes and normalized gene scores.

3.4.2 Algorithm

Score Computation Steps:

- *1. Identify beneficial genes: filter compound targets where beneficial count ≥ 1 in disease signature*

- 2. Lookup scores: for each beneficial gene, retrieve normalized score from Excel 'Final Gene Score' sheet
- 3. Aggregate: sum all individual gene scores
- 4. Normalize: divide aggregate by constant divisor (283.4119365)
- 5. Return: final repurposing score suitable for ranking and comparison

3.4.3 Functional Requirements

- FR-4.1: System shall identify beneficial genes (count ≥ 1 in disease signature)
- FR-4.2: System shall sum Final Gene Score values from Excel lookup table for all beneficial genes
- FR-4.3: System shall normalize aggregate score by divisor constant 283.4119365
- FR-4.4: System shall return final repurposing score with 4-decimal precision for ranking and comparison
- FR-4.5: System shall cache score computations for identical gene-disease pairs within single session

3.4.4 Interfaces

- API: GET /api/finalGeneScore (returns computed score)
- UI: Score displayed prominently in GeneMatchPage with color gradient (red=low, yellow=medium, green=high)

3.5 Gene Expression and Dataset Exploration

3.5.1 Description

This feature provides interactive browsing of gene expression metadata and other tabular datasets through paginated, searchable interfaces to support exploratory analysis.

3.5.2 Functional Requirements

- FR-5.1: System shall support paginated search of gene expression data from GeneCards Excel with configurable page size
- FR-5.2: System shall allow text search on gene names and identifiers with real-time filtering
- FR-5.3: System shall export disease signature tables with up-regulated and down-regulated genes to JSON format
- FR-5.4: System shall provide full dataset metadata (sheet names, row counts, column headers) through dedicated API endpoint
- FR-5.5: System shall cache Excel workbook reads in memory to avoid repeated disk I/O for large datasets

3.5.3 Interfaces

- API: GET /api/geneExpressions?page=0&page_size=10&search=query (paginated gene search)
- API: GET /api/excelData/meta (returns metadata for all Excel sheets)
- API: GET /api/excelData/sheet?name=sheetname&page=0&page_size=50 (returns paginated sheet data)
- UI: GeneExpressionsPage and ExcelViewerPage with search boxes, pagination controls, and data tables

3.6 Data Export and Results Persistence

3.6.1 Description

This feature allows researchers to export analysis results and intermediate data for further processing in external tools.

3.6.2 Functional Requirements

- FR-6.1: System shall export computed disease signature tables to local filesystem in JSON format
- FR-6.2: System shall support JSON export of match results including gene classifications and final score
- FR-6.3: System shall persist session state (selected compound, disease, results) for user navigation between interface views
- FR-6.4: System shall include metadata in exported files (timestamp, compound name, disease name, score computation details)
- FR-6.5: System shall support download of results through browser as JSON attachment

3.6.3 Interfaces

- UI: Export buttons on GeneMatchPage and GeneExpressionsPage
- API: Results endpoints support 'download=true' query parameter to trigger file download

4. External Interface Requirements

4.1 User Interfaces

4.1.1 HomePage

Primary entry point for compound search. Displays:

- Title and brief description of ReMediX functionality
- Two search input fields (compound name or SMILES)

- Search button and recent searches (if applicable)
- Results area showing compound properties (name, image, molecular weight, InChIKey)
- Action buttons to navigate to GeneMatchPage with extracted gene targets

4.1.2 GeneMatchPage

Disease selection and gene matching. Displays:

- Selected compound name and key properties
- Extracted gene targets from compound (from ChEMBL)
- Autocomplete disease selection dropdown
- Gene matching results (beneficial/harmful counts, gene lists)
- Computed repurposing score with interpretation guidance
- Disease signature table (paginated, sortable)
- Export buttons for results and signature table

4.1.3 GeneExpressionsPage

Gene expression data explorer. Displays:

- Search box for gene name/identifier filtering
- Paginated table of gene expression data from GeneCards
- Pagination controls (previous, next, jump to page)
- Page size selector (10, 25, 50 rows per page)
- Column headers with optional sorting

4.1.4 ExcelViewerPage

Tabular dataset browser. Displays:

- Dropdown to select which Excel sheet to view
- Metadata display (sheet name, total rows, column headers)
- Paginated data table with configurable page size
- Search/filter functionality across visible columns

4.1.5 PPIInteractionPage

Protein-protein interaction visualization (placeholder for future expansion). Currently displays:

- Brief description of planned PPI analysis capabilities
- Placeholder interface for future feature implementation

4.2 Hardware Interfaces

ReMediX requires standard computing hardware:

- Display: Monitor with minimum 1920x1080 resolution (recommended for data-intensive interfaces)
- Input: Keyboard and mouse/trackpad for navigation

- Network: Ethernet or WiFi adapter for API connectivity
- Storage: Local disk for dataset files (minimum 2GB available space)
- Processing: Multi-core processor (2+ cores) for concurrent request handling
- Memory: Minimum 4GB RAM for single-user sessions; 8GB+ for concurrent users

4.3 Software Interfaces

4.3.1 PubChem REST API

Public API for compound property lookups. Integration details:

- Base URL: <https://pubchem.ncbi.nlm.nih.gov/rest/pug/>
- Endpoints used: `/compound/{identifier}/property/`, `/compound/{identifier}/JSON`
- Request format: HTTP GET with URL parameters
- Response format: JSON
- Error handling: Returns 404 for unknown compounds; system returns user-friendly error message
- Rate limiting: No explicit rate limit documented; system implements reasonable caching to minimize requests

4.3.2 ChEMBL webresource-client

Python library for ChEMBL API access. Integration details:

- Library: `chembl-webresource-client` (installed via `pip`)
- Initialization: Lazy loading on first use (not at application startup)
- Methods used: `activity.filter(molecule_chembl_id=X, standard_type=Y)`, `target.filter(organism='Homo sapiens')`
- Caching: LRU cache implemented at function level (`@lru_cache`)
- Error handling: Timeouts and API errors return empty results with warning logs

4.3.3 Local DrugBank Dataset

Local XML file for pharmacology lookups. Integration details:

- Format: XML (DrugBank release format)
- Location: `src/data/drugbank.xml` (configurable via environment variable)
- Parsing: Streaming XML parser (`lxml.etree.iterparse`) to avoid memory overhead
- Lookup key: InChIKey matching against drugbank record identifiers

- Error handling: Missing file raises critical error; malformed XML logged and skipped

4.3.4 CREEDS Disease Signatures

Local JSON file for disease gene sets. Integration details:

- Format: JSON (custom structure with disease name, up-genes, down-genes)
- Location: `src/data/CREEDS/disease_signatures-v1.0.json`
- Loading: Full load into memory at startup (file size ~100MB)
- Lookup: Case-insensitive disease name matching
- Error handling: Missing file raises critical error; malformed JSON raises validation error

4.3.5 Excel Data Files

Local Excel spreadsheets for scoring and gene data. Integration details:

- Format: XLSX (Microsoft Excel 2007+ format)
- Files: Final Gene Score lookup table, GeneCards expression data
- Location: `src/data/` directory (configurable)
- Parsing: `openpyxl` library for reading
- Caching: Entire workbook loaded into memory on first access with LRU caching
- Error handling: Missing files raise errors; missing columns logged as warnings

4.4 Communications Interfaces

ReMediX communicates via standard HTTP/HTTPS protocols:

- Backend API Server: FastAPI application listening on configurable host:port (default localhost:8000)
- Frontend Access: Browser accesses backend via fetch API (XMLHttpRequest wrapper)
- External APIs: HTTPS requests to PubChem and ChEMBL endpoints
- CORS: Frontend dev mode uses Vite proxy to avoid CORS issues; production uses same-origin or configured CORS headers
- Data Format: All internal and external communication via JSON (except legacy XML parsing for DrugBank)
- Authentication: None implemented (internal use assumed); token-based auth can be added via middleware

5. Other Non-Functional Requirements

5.1 Performance Requirements

- NFR-1.1: Compound lookup from PubChem shall complete in <5 seconds with timeout handling
- NFR-1.2: Local DrugBank lookup shall complete in <2 seconds using streamed XML parsing
- NFR-1.3: ChEMBL bioactivity enrichment shall complete in <10 seconds with caching enabled
- NFR-1.4: Disease matching and score computation shall complete in <1 second (local operations)
- NFR-1.5: Pagination queries shall complete in <500 ms
- NFR-1.6: System shall handle 5-10 concurrent users without performance degradation
- NFR-1.7: In-memory caching shall reduce repeated query latency by 90% vs. first execution

5.2 Safety Requirements

Safety considerations for biomedical research context:

- NFR-2.1: System shall not lose or corrupt locally stored datasets during operation
- NFR-2.2: System shall validate all external data (PubChem, ChEMBL) for consistency before use
- NFR-2.3: System shall handle missing or incomplete data gracefully with informative warnings
- NFR-2.4: System shall not make irreversible changes to source datasets without explicit user action
- NFR-2.5: Repurposing scores shall include confidence indicators (e.g., based on gene set size) for researcher awareness

5.3 Security Requirements

- NFR-3.1: System is intended for controlled internal use; no authentication/authorization required
- NFR-3.2: External API credentials (if any) shall be managed via environment variables, not hardcoded
- NFR-3.3: Input validation shall prevent injection attacks (query parameters, file paths)
- NFR-3.4: File paths shall be sanitized to prevent directory traversal attacks
- NFR-3.5: If exposed to untrusted networks, HTTPS and token-based auth must be added

- *NFR-3.6: System shall log access to external APIs (for future audit trail if needed)*
- *NFR-3.7: System shall not expose internal error details to frontend (generic error messages preferred)*

5.4 Software Quality Attributes

- *Maintainability: Code organized by client (one per data source); easy to add/update data sources*
- *Testability: API endpoints designed to be independently testable; dependencies injectable for mocking*
- *Usability: Intuitive multi-step workflow; clear error messages guide users to resolution*
- *Portability: Cross-platform (Linux, macOS, Windows WSL2) with containerization support via Docker*
- *Reliability: Graceful degradation if external APIs unavailable; no single point of failure for local operations*
- *Modifiability: Loosely coupled architecture allows updates to individual components without full system rebuild*

6. Other Requirements

6.1 Build and Deployment

Backend build: 'pip install -r requirements.txt' installs Python dependencies. Frontend build: 'npm run build' compiles React SPA to /frontend/dist. Production deployment bundles SPA with backend for serving as single process.

6.2 Installation and Setup

Users must provision local datasets (DrugBank XML, CREEDS JSON, Excel files) in src/data/ directory before first run. Detailed setup instructions provided in Getting Started Guide.

6.3 Regulatory and Compliance

No regulatory compliance requirements beyond academic data usage policies. Datasets subject to their respective licenses (DrugBank, ChEMBL, CREEDS). Users responsible for license compliance.

6.4 Standards and Conventions

- *Code follows PEP 8 (Python) and ESLint (JavaScript) standards*
- *API follows RESTful conventions with consistent naming (snake_case for parameters)*
- *Data exchange via JSON with OpenAPI/Swagger documentation*

- *Git-based version control with semantic versioning*

Appendix A: Glossary of Terms

<i>Term</i>	<i>Definition</i>
<i>Bioactivity</i>	<i>Quantitative measure of interaction between compound and biological target (IC50, AC50, Ki)</i>
<i>CREEDS</i>	<i>Curated gene sets representing disease signatures (up-regulated/down-regulated genes)</i>
<i>DrugBank</i>	<i>Comprehensive database of drug information, properties, and mechanisms of action</i>
<i>ChEMBL</i>	<i>Chemical database with bioactivity data linking compounds to biological targets</i>
<i>InChIKey</i>	<i>Standardized hash of chemical structure; enables cross-database compound identity</i>
<i>Gene Target</i>	<i>Protein or gene whose expression/activity is affected by a compound</i>
<i>Repurposing Score</i>	<i>Quantitative metric indicating likelihood of compound efficacy for target disease</i>
<i>SMILES</i>	<i>Simplified Molecular Input Line Entry System; text notation for chemical structure</i>
<i>PubChem</i>	<i>Free compound database from NCBI with chemical identifiers and properties</i>
<i>GeneCards</i>	<i>Gene expression and genomic data repository for human genes</i>

Appendix B: Analysis Models

B.1 Data Flow Diagram - Compound Search to Scoring

High-level data flow:

- *User Input (compound name/SMILES) → PubChem Lookup → InChIKey Resolution*
- *InChIKey → DrugBank Lookup (local) + ChEMBL Lookup (external)*
- *DrugBank + ChEMBL → Gene Target Set Extraction*
- *Gene Targets + Disease Selection → CREEDS Matching (local)*
- *Gene Classification → Final Gene Score Lookup (Excel) → Score Normalization*
- *Final Score + Results → Display/Export*

B.2 System Integration Architecture

Components and their interactions:

- *Frontend (React): User-facing interfaces, form inputs, result display, navigation*
- *Backend API (FastAPI): Routes requests to appropriate client modules*
- *Client Modules: pubchem_client, drugbank_client, chembl_client, creeds_client, geneCards_client, final_gene_score*
- *Data Layer: Local files (XML, JSON, XLSX) loaded with caching; External APIs (PubChem, ChEMBL)*

B.3 Caching and Performance Strategy

Multi-level caching reduces latency and API calls:

- *LRU Cache: Function-level caching for ChEMBL queries (@lru_cache decorator)*
- *In-Memory Cache: Entire DrugBank/CREEDS/Excel datasets loaded once at startup*
- *Browser Cache: Frontend leverages HTTP caching for static assets and API responses*
- *Session Cache: Results within single user session cached to avoid re-computation*

Appendix C: Issues List and Open Questions

C.1 Known Issues

C.1.1 API Contract Inconsistency

Some pagination endpoints use 'page_size' (snake_case) while others may use 'pageSize'. Frontend fetch wrapper should normalize field names. Priority: Medium. Resolution: Standardize to snake_case in v1.1.

C.1.2 String-Encoded Match Counts

Gene match counts returned as strings ('beneficial: 5') rather than numeric JSON fields. Downstream parsing is brittle. Priority: Low (works but inelegant). Resolution: v2 API to use numeric fields.

C.1.3 Synchronous Filesystem Writes

Disease signature table export writes synchronously during request handling. Concurrent requests may contend. Priority: Medium. Resolution: Implement write queue or cache tables in memory.

C.1.4 No Automated Test Suite

System validation is manual; no unit/integration tests detected. Regression risk high. Priority: High. Resolution: Build test harness before production release.

C.2 Recommendations for Future Enhancements

- Implement user accounts and result persistence (requires database backend)
- Add machine learning scoring refinement based on known drug-disease pairs
- Integrate PPI network visualization for target validation
- Support batch processing of multiple compounds/diseases
- Implement real-time collaboration for research teams
- Add literature mining integration for supporting evidence

--- END OF DOCUMENT ---